Collaborative Interactive Shell (CIS) — Phase 2 Implementation, Demonstration, and Design Evolution

Reza Kashkoul
rezakashkoul@gmail.com

March 2026

1. Overview

This document describes the Phase 2 implementation of a Collaborative Interactive Shell (CIS), extending the Phase 1 proposal into a working shared terminal environment.

The system allows multiple already-authenticated users on the same machine to attach to one shared shell session. At any time:

- exactly one client is the **Controller**
- all other clients are **Observers**

The Controller's input is forwarded to the shell. Observers see the same live output, but their ordinary input does not reach the shell.

The implementation remains aligned with the Phase 1 design:

- existing shell foundation: `/bin/bash`
- shell attached to a PTY
- local client/server communication through a Unix domain socket
- single-threaded event-loop design

2. Phase 1 to Phase 2 Evolution

Phase 1 validated the general architecture using a PTY-backed shell and a minimal prototype host. It also established the intended floor-control model and fairness policy.

The main Phase 2 evolution was to replace the Phase 1 prototype interaction model with:

- an explicit client program
- a separate control path and shell-input path
- FIFO request handling with one outstanding request per client
- practical disconnect and cleanup behavior
- controller-driven terminal resize propagation

This design evolution directly addresses the feedback received in Phase 1 regarding:

- implementation of request/release
- use of a simpler escape mechanism such as `Ctrl-T`
- precise meaning of an outstanding request

3. Foundation

3.1. Exact foundation components

- OS: Fedora Linux 43 (Workstation Edition)
- Kernel: Linux 6.17.12-300.fc43.x86_64
- Shell: `/bin/bash`, GNU bash 5.3.0(1)-release

3.2. What the foundation provides

Using an existing shell attached to a PTY provides:

- interactive shell behavior
- job control and built-in shell semantics
- prompt and terminal interaction expected by a normal shell

3.3. What CIS implements

The collaboration layer implemented in this project provides:

- multiple client attach
- real-time output broadcast
- controller/observer role separation
- request/release/cancel logic
- participant inspection
- kick support
- disconnect handling
- reconnect as a fresh client connection
- resize propagation from the controller to the shared PTY
- cleanup on server shutdown

4. System Architecture

The system contains two main programs.

4.1. `cis_server.py`

The server:

- creates a PTY-backed shell
- accepts Unix domain socket client connections
- broadcasts shell output to all clients
- tracks controller state
- enforces floor-control policy
- handles kick, resize, disconnects, queue updates, and cleanup

4.2. `cis_client.py`

The client:

- connects to the server
- displays shared output
- forwards shell input
- provides a `Ctrl-T` command mode for control operations
- detects `SIGWINCH` and reports controller resize changes

5. Floor-Control Model

5.1. Roles

- **Controller**: shell input is forwarded
- **Observer**: shell input is rejected, but shared output is visible

5.2. Request / Grant / Revoke

The client uses `Ctrl-T` to enter command mode. Supported control commands are:

```
request
release
cancel
who
name NEWNAME
kick ID
quit
```

Request control: Observers may request control through command mode.
Grant control: Control is granted according to FIFO order.
Revoke control: Control may be revoked by:

- explicit `release`
- controller disconnect
- server shutdown

5.3. Fairness policy

The fairness policy is **FIFO**.

Each client may have at most one outstanding request at a time. A duplicate request is ignored until the previous request has been:

- granted
- canceled
- removed because of disconnect

This resolves the ambiguity raised in the Phase 1 feedback about what counts as an outstanding request and how request spamming is prevented.

6. Protocol Design

One major implementation improvement in Phase 2 was the explicit separation of:

- control commands
- ordinary shell input

6.1. Control messages

The client sends control actions as:

```
CONTROL request
CONTROL release
CONTROL cancel
CONTROL who
CONTROL name NEWNAME
CONTROL kick ID
CONTROL resize ROWS COLS
```

6.2. Shell input messages

Normal shell input is sent separately as:

```
INPUT <hex-encoded-bytes>
```

This design avoids mixing shell input with control commands and improved interactive correctness compared to an earlier line-oriented approach.

7. Resize Policy

Resize support is implemented with a simple controller-owned policy.

- Only the current controller may resize the shared PTY.
- Observer-side window resizing remains local to that observer's terminal.
- Observer resize is not forwarded to the shared PTY.

The implementation uses:

- `SIGWINCH` in the client
- terminal size discovery via `TIOCGWINSZ`
- resize application at the PTY using `TIOCSWINSZ`

This keeps resize semantics simple and avoids conflicting terminal sizes from multiple observers.

8. Implemented Features

The final implementation supports:

- host creates a shared interactive shell
- one or more clients attach
- live output broadcast to all clients
- exactly one controller at a time
- request/release/cancel
- participant listing through `who`
- client rename
- kick by controller
- observer read-only enforcement
- controller disconnect handling
- observer exit handling
- practical reconnect as a fresh client connection
- controller-driven PTY resize propagation
- server shutdown cleanup

9. OS Mechanisms Used

This implementation uses the same OS-level ideas identified in Phase 1.

9.1. PTY-backed shell

The shell runs inside a PTY, preserving interactive terminal behavior.

9.2. Process management

The implementation uses:

- fork/exec to launch the shell
- `waitpid` to reap the child
- signals for shutdown

9.3. I/O multiplexing

The server uses a single-threaded event loop with `select()` across:

- PTY master
- server socket
- connected client sockets

The client also uses `select()` across:

- stdin
- connected socket

9.4. Terminal handling

The client places the terminal into raw mode to:

- capture keystrokes directly
- implement `Ctrl-T` command mode
- distinguish observer and controller handling for `Ctrl-C`
- detect window resize events

10. Failure Handling

10.1. Controller disconnect

If the controller disconnects, the shell remains alive. The next queued requester becomes controller automatically. If no queued requester exists, the session remains alive with no controller assigned.

10.2. Duplicate requests

A client cannot enter the request queue multiple times.

10.3. Observer exit

An observer may exit without disrupting the shared shell session.

10.4. Reconnect / reattach behavior

This implementation supports practical reconnect as a fresh new client connection. A reconnecting client receives a new client id and joins as an observer unless or until floor-control policy grants control.

This means reconnect is supported functionally, but identity-preserving reattachment is not implemented.

10.5. Network drop / reconnect interpretation

Because this implementation uses a same-machine Unix domain socket rather than a remote network transport, "network drop / reconnect" is modeled as client disconnect followed by a fresh reconnect. Packet-level remote network fault handling is not implemented.

10.6. Server shutdown

When the server shuts down:

- all client connections are closed
- the Unix socket path is removed
- the shell child is signaled and reaped
- the PTY descriptor is closed

10.7. Ctrl-C behavior

The client handles `Ctrl-C` as follows:

- in command mode, it cancels command mode
- if the client is an observer, it exits the client
- if the client is the controller, it is forwarded to the shell

11. Testing Summary

The following behaviors were validated through manual interactive testing and terminal transcripts collected during development:

- initial controller assignment
- observer read-only enforcement
- FIFO request queue
- duplicate request rejection
- controller release and promotion of next requester
- controller disconnect and automatic promotion
- observer exit with `Ctrl-C`
- controller sending `Ctrl-C` to the shell
- controller kick of observer
- observer rejection when attempting kick
- server shutdown cleanup
- fresh reconnect behavior
- controller-driven resize propagation

12. Example Usage

12.1. Start the server

```
cd /home/rkashko/os/project1
rm -f cis.sock
python3 cis_server.py ./cis.sock
```

12.2. Start a client

```
cd /home/rkashko/os/project1
python3 cis_client.py ./cis.sock
```

12.3. Enter command mode

```
Ctrl-T
```

12.4. Request control

```
Ctrl-T
request
```

12.5. Release control

```
Ctrl-T
release
```

12.6. Inspect CIS participant state

```
Ctrl-T
who
```

12.7. Kick a client

```
Ctrl-T
kick 2
```

13. Important Clarification

There is an important distinction between:

- command-mode `who`
- shell command `who`

Command-mode `who` displays CIS participant and queue state.
Shell `who` runs the normal system command inside the shared shell.
These should not be confused during demonstration.

14. Hardest Bugs Encountered

14.1. Output alignment issue

Because the client used raw terminal mode, newline handling initially produced visibly misaligned output. This was fixed by normalizing newline sequences before printing received output.

14.2. Line-buffered input issue

An earlier version treated input too much like line-based text, which interfered with correct interactive typing. This was corrected by explicitly separating:

- control commands
- ordinary shell input

into distinct protocol messages.

15. Limitations

This implementation intentionally focuses on the assignment core. The following are not fully implemented:

- identity-preserving reconnect
- persistent ban support
- advanced multi-client resize arbitration beyond the controller-owned PTY size
- remote network fault handling beyond local disconnect/reconnect semantics

Kick is implemented, but ban is not maintained as a persistent policy.

16. Artifacts Submitted

Phase 2 artifacts include:

- `cis_server.py`
- `cis_client.py`
- `README.md`
- `demo_transcript.txt`
- `POSTMORTEM.txt`
- `trace/`

17. Conclusion

This Phase 2 implementation successfully extends the Phase 1 proposal into a working collaborative shell system with:

- PTY-backed shared shell execution
- explicit controller/observer separation
- FIFO floor control
- practical reconnect behavior
- kick support
- controller-driven resize propagation

- disconnect robustness
- session cleanup

The final design remains consistent with the original Phase 1 proposal while improving control-path clarity, robustness, and practical usability.